

ECE327- Ψηφιακά Συστήματα VLSI

Εργαστηριακή Εργασία Εξαμήνου - Μέρος 2ο Σχεδίαση, Χαρακτηρισμός και Χρήση Standard Cells Πέτρογλου Σπυρίδων 03185

Στόχος: Δημιουργία βιβλιοθήκης σύμφωνα με το Synopsys Liberty Standard

Στα πλαίσια της εργασίας μου, θα πρέπει να δημιουργήσω μια βιβλιοθήκη που θα περιέχει χρήσιμες πληροφορίες για μερικά ψηφιακά κυκλώματα (standard cells): τις λογικές πύλες XOR/XNOR και τα ακολουθιακά κυκλώματα DFF/DFFRS. Αυτό θα συμβεί δημιουργώντας ένα εργαλείο (σε γλώσσα python), το οποίο θα λαμβάνει τα δεδομένα του από ένα configuration file (σε γλώσσα json), και θα παράγει την βιβλιοθήκη με κατάληξη lib.

Το configuration file είναι γραμμένο σε γλώσσα JSON, καθώς θεωρείται η ευκολότερη γλώσσα για διαχείριση δεδομένων, ειδικά δεδομένων που θα διαβαστούν στην συνέχεια από ένα πρόγραμμα python. Περιέχει έναν πίνακα με όνομα models, που περιέχει όλα τα standard cells που επρόκειτο να συμπεριληφθούν στην βιβλιοθήκη. Η κάθε θέση του πίνακα, είναι ένα αντικείμενο που περιέχει ως κλειδιά το όνομα του εκάστοτε standard cell, τον τύπο του κυκλώματος (εάν είναι συνδυαστικό ή ακολουθιακό), την boolean συνάρτηση εξόδου και το filepath στο spice deck. Εκτός από αυτές τις πληροφορίες, το αντικείμενο περιέχει επίσης τα pins εισόδου μέσα σε έναν πίνακα, τα pins εξόδου σε έναν άλλον πίνακα, και τα timing/constraint arcs σε άλλους πίνακες. Ένα μέρος από το configuration file με όλα τα παραπάνω φαίνεται στην παρακάτω εικόνα

```
1  {
2      "models": [
3          {
4              "name": "XOR2_X1",
5              "type": "combinational",
6              "function": "(A ^ B)",
7              "filepath": "spice_exercises/semester_project/xor.spice",
8              "input_pins": ["in1xor", "in2xor"],
9              "output_pins": ["xor_out"],
10             "timing_arcs": [
11                 {
```

Οι πίνακες των timing και των constraint arcs, περιέχουν αντικείμενα με τις εξής πληροφορίες: Το άμεσα συσχετιζόμενο pin εισόδου με το timing arc (ή αλλιώς, από που «ξεκινάει» το timing arc), το pin εξόδου στο οποίο καταλήγει το timing arc, την μονοτονία (unateness) του arc, την τιμή πλευρικής εισόδου (πολύ σημαντική πληροφορία εάν η πύλη ή το κύκλωμα είναι binate), ποιο template χρησιμοποιούμε για τον χαρακτηρισμό των arcs (τα templates παρουσιάζονται παρακάτω) και τα filepath που οδηγούν στα αρχεία με τις μετρήσεις (θα παρουσιαστούν και αυτά παρακάτω αναλυτικότερα). Ένα στιγμιότυπο από οκτώ εκ των, συνολικά δεκαέξι, timing arcs της πύλης XOR, φαίνεται παρακάτω. Τα timing arcs ομαδοποιούνται ανάλογα με το περιεχόμενό τους. Για παράδειγμα, αυτά που έχουν κοινό pin εισόδου, κοινό pin εξόδου και κοινό side input, μπαίνουν στο ίδιο index του πίνακα. Όπως είναι λογικό, το κάθε index δεν πρέπει να περιέχει πάνω από μια φορά το ίδιο arc.

```

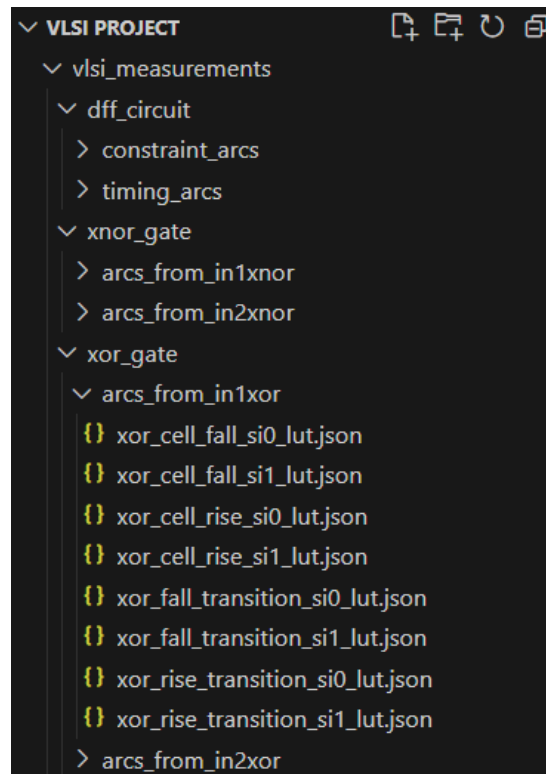
10 "timing_arcs": [
11 {
12     "from_pin": "in1xor",
13     "to_pin": "xor_out",
14     "side_input": 0,
15     "unateness": "positive_unate",
16     "lut_template": "Timing_7_7",
17     "cell_rise": "vlsi_measurements/xor_gate/arcs_from_in1xor/xor_cell_rise_si0_lut.json",
18     "cell_fall": "vlsi_measurements/xor_gate/arcs_from_in1xor/xor_cell_fall_si0_lut.json",
19     "rise_transition": "vlsi_measurements/xor_gate/arcs_from_in1xor/xor_rise_transition_si0_lut.json",
20     "fall_transition": "vlsi_measurements/xor_gate/arcs_from_in1xor/xor_fall_transition_si0_lut.json"
21 },
22 {
23     "from_pin": "in2xor",
24     "to_pin": "xor_out",
25     "side_input": 0,
26     "unateness": "positive_unate",
27     "lut_template": "Timing_7_7",
28     "cell_rise": "vlsi_measurements/xor_gate/arcs_from_in2xor/xor_cell_rise_si0_lut.json",
29     "cell_fall": "vlsi_measurements/xor_gate/arcs_from_in2xor/xor_cell_fall_si0_lut.json",
30     "rise_transition": "vlsi_measurements/xor_gate/arcs_from_in2xor/xor_rise_transition_si0_lut.json",
31     "fall_transition": "vlsi_measurements/xor_gate/arcs_from_in2xor/xor_fall_transition_si0_lut.json"
32 },

```

Τα timing arcs που χαρακτηρίστηκαν είναι η καθυστέρηση πύλης (cell or propagation delay), που χωρίζεται σε cell rise και cell fall, και η καθυστέρηση μετάβασης εξόδου (cell -output- transition) ή κλίση εξόδου (output slew), που και αυτήν χωρίζεται σε rise transition και fall transition. Τα constraint arcs που επρόκειτο να χαρακτηριστούν είναι τα setup_rising και hold_rising για το DFF, ενώ για το DFFRS πρέπει να χαρακτηριστούν οι προαναφερόμενοι περιορισμοί, και επιπλέον τα recovery_rising και removal_rising.

Για τον χαρακτηρισμό των συσχετίσεων, θα χρησιμοποιηθεί το NGspice και το NLDM model. Σύμφωνα με το NLDM model, οι καθυστερήσεις μοντελοποιούνται ως συναρτήσεις 2 μεταβλητών, με ανεξάρτητες μεταβλητές την κλίση εισόδου (input slew) και την χωρητικότητα εξόδου (output load). Οι καθυστερήσεις μετριοούνται με την χρήση του NGspice. Από την στιγμή που έχουμε 2 ανεξάρτητες μεταβλητές, μπορούμε να αποθηκεύσουμε τα αποτελέσματα, δηλαδή τις καθυστερήσεις, σε έναν διδιάστατο πίνακα, τα λεγόμενα Look-Up-Tables (LUTs). Οπότε θα χρειαστεί να δημιουργηθεί ένα LUT για κάθε arc που επρόκειτο να χαρακτηριστεί.

Τα LUTs είναι τοποθετημένα σε διαφορετικά αρχεία από το configuration file, αλλά σε αυτό συμπεριλαμβάνονται τα filepaths αυτών των αρχείων, όπως φαίνεται στην εικόνα επάνω. Προφανώς, ένα αρχείο περιέχει ακριβώς ένα LUT. Επειδή τα LUTs ήταν πάρα πολλά, έπρεπε σε αυτά τα αρχεία να δωθούν χαρακτηριστικά ονόματα για το ποιο arc χαρακτηρίζουν, αλλά και να οργανωθούν κατάλληλα σε φακέλους. Για παράδειγμα, παρακάτω φαίνονται τα ονόματα των LUTs για την πύλη XOR που περιγράφουν όλα τα timing arcs που ξεκινούν από το in1xor pin. Παρατηρούμε ότι για να είναι ευδιάκριτο το ποιο arc περιέχεται σε ένα αρχείο, το όνομα του αρχείου πρέπει να περιλαμβάνει το όνομα του standard cell, το όνομα του arc και την τιμή του side input.



Μέσα στα αρχεία αυτά, περιέχονται οι τιμές των καθυστερήσεων. Τα αρχεία αυτά περιέχουν έναν πίνακα που ονομάζεται measurements, με τις θέσεις του να είναι αντικείμενα, όπου το κάθε αντικείμενο περιγράφει μια στήλη ενός LUT. Η δομή μιας στήλης του αρχείου xor_cell_fall_si0_lut.json φαίνεται παρακάτω

```

1  {
2      "measurements": [
3          {
4              "input_slew": 0.00231025,
5              "output_load": [
6                  {
7                      "output_load": 0.365616,
8                      "value": 1.444798
9                  },
10                 {
11                     "output_load": 0.791550,
12                     "value": 1.445786
13                 },
14                 {
15                     "output_load": 1.583100,
16                     "value": 1.447621
17                 },
18                 {
19                     "output_load": 3.166200,
20                     "value": 1.451283
21                 },
22                 {
23                     "output_load": 6.332400,
24                     "value": 1.458627
25                 },
26                 {
27                     "output_load": 12.664800,
28                     "value": 1.473118
29                 },
30                 {
31                     "output_load": 25.329600,
32                     "value": 1.501658
33                 }
34             ]
35         },

```

Τέλος, τα template που χρησιμοποιήθηκαν για τον χαρακτηρισμό των arcs είναι το Timing_7_7 της βιβλιοθήκης Nangate για τα timing arcs, και τα Setup_3_3, Hold_3_3, Recovery_3_3, Removal_3_3 για τα constraint arcs, με τις αρχικοποιήσεις τους να φαίνονται παρακάτω.

```
lu_table_template (Hold_3_3) {
    variable_1 : constrained_pin_transition;
    variable_2 : related_pin_transition;
    index_1 ("0.0010,0.0020,0.0030");
    index_2 ("0.0010,0.0020,0.0030");
}

lu_table_template (Recovery_3_3) {
    variable_1 : constrained_pin_transition;
    variable_2 : related_pin_transition;
    index_1 ("0.0010,0.0020,0.0030");
    index_2 ("0.0010,0.0020,0.0030");
}

lu_table_template (Removal_3_3) {
    variable_1 : constrained_pin_transition;
    variable_2 : related_pin_transition;
    index_1 ("0.0010,0.0020,0.0030");
    index_2 ("0.0010,0.0020,0.0030");
}

lu_table_template (Setup_3_3) {
    variable_1 : constrained_pin_transition;
    variable_2 : related_pin_transition;
    index_1 ("0.0010,0.0020,0.0030");
    index_2 ("0.0010,0.0020,0.0030");
}

lu_table_template (Timing_7_7) {
    variable_1 : input_net_transition;
    variable_2 : total_output_net_capacitance;
    index_1 ("0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070");
    index_2 ("0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070");
}
```

ΠΥΛΗ XOR

Η πύλη XOR είναι μία λογική πύλη που κατασκευάζεται με βάση την συνδιαστική λογική. Το πιο ευρέως διαδεδομένο μοντέλο της αποτελείται από 2 εισόδους και 1 έξοδο, και ονομάζεται XOR2_X1 (όπου X1 η οδηγητική ικανότητα της πύλης, δεν μας αφορά καθώς όλες οι πύλες είναι σχεδιασμένες να έχουν οδηγητική ικανότητα 1 στα πλαίσια της εργασίας). Η λογική της είναι: όταν το ένα pin εισόδου είναι 1 και το άλλο 0, το pin εξόδου είναι 1. Διαφορετικά, εάν δηλαδή και τα 2 pin εισόδου είναι 0 ή 1, η έξοδός της είναι 0. Αυτό σημαίνει ότι εάν μεταβάλλουμε ένα bit εισόδου, τότε η έξοδος αλλάζει. Για παράδειγμα, έστω ότι έχουμε ως είσοδο τα bits 01. Η έξοδός της θα είναι λογικό 1. Εάν μεταβάλλω το 1^ο bit θα έχω ως

νέα είσοδο το 11 ενώ εάν μεταβάλλω το 2^ο bit θα έχω ως είσοδο το 00, δηλαδή λογικό 0. Αντίστοιχα αποτελέσματα θα έχω εάν ξεκινήσω από bits εισόδου 00 ή 11. Για αυτόν τον λόγο, η μονοτονία (unateness) της πύλης χαρακτηρίζεται ως Binate, το οποίο σημαίνει εάν έχω είσοδο «ράμπας» (δηλαδή μεταβολή ενός bit) σε κάποιο pin εισόδου, δεν μπορώ να γνωρίζω την «ράμπα» που θα έχω στην έξοδο. Για να μάθω την «ράμπα», θα πρέπει να γνωρίζω και την τιμή της πλευρικής εισόδου (side input) της πύλης.

Οπότε, προκύπτουν 16 timing arcs για την πύλη XOR, 8 από αυτά αφορούν το cell delay και τα άλλα 8 το output transition, τα οποία ομαδοποιούνται σε τετράδες (εικόνα σελ. 2).

Οι τιμές input slew και output load που χρησιμοποιήθηκαν υπάρχουν στο μοντέλο Timing_7_7 της βιβλιοθήκης Nangate και φαίνονται παρακάτω (οι ίδιες τιμές ισχύουν για όλα τα timing arcs της πύλης)

```
cell_rise(Timing_7_7) {
    index_1 ("0.00231025,0.0112628,0.0426883,0.102700,0.196195,0.327379,0.500000");
    index_2 ("0.365616,0.791550,1.583100,3.166200,6.332400,12.664800,25.329600");
```

Τα αρχεία spice που χρησιμοποίησα για να χαρακτηρίσω όλα τα timing arcs της πύλης είναι τα:

- xor_inputs_rising_in1.spice
- xor_inputs_rising_in2.spice
- xor_inputs_falling_in1.spice
- xor_inputs_falling_in2.spice

Σε κάθε αρχείο χαρακτηρίζω 4 timing arcs (2 cell delays και 2 output transitions για side inputs 0 και 1). Οπότε, σε κάθε αρχείο πρέπει να επιλεχθούν σωστά 3 πράγματα: Το input slew, το output load και το side input. Πολύ σημαντικό το καθένα από αυτά να έχει επιλεχθεί ακριβώς μία φορά! Επίσης, κάθε αρχείο περιέχει και measurement commands έτσι ώστε να εμφανίζονται οι τιμές των καθυστερίσεων στο terminal. Υπάρχουν επίσης αναλυτικές οδηγίες στα σχόλια των αρχείων.

```
*****
***This file contains the delay measurements while in1 is rising and in2 is constant
*****
***Firstly, let's define input slew 30%-70% at 0.5ns at in1 while in2=const
***Last column of the LUT, for the xor gate
***Unfortunately, I need to define and the complementary values in3, in4

*V1xor in1xor 0 PWL (0 0 3.75n 0 5n 2.5 10n 2.5)
*V3xor in3xor 0 PWL (0 2.5 3.75n 2.5 5n 0 10n 0)

*****
***Now, it's time to give the value 0.327379ns at my input slew, again in2=const

*V1xor in1xor 0 PWL (0 0 4.1815525n 0 5n 2.5 10n 2.5)
*V3xor in3xor 0 PWL (0 2.5 4.1815525n 2.5 5n 0 10n 0)

*****
***Third value: 0.196195ns at my input slew, again in2=const

*V1xor in1xor 0 PWL (0 0 4.5095125n 0 5n 2.5 10n 2.5)
*V3xor in3xor 0 PWL (0 2.5 4.5095125n 2.5 5n 0 10n 0)

*****
***Fourth value: 0.102700ns at my input slew, again in2=const

*V1xor in1xor 0 PWL (0 0 4.74325n 0 5n 2.5 10n 2.5)
*V3xor in3xor 0 PWL (0 2.5 4.74325n 2.5 5n 0 10n 0)
```

```

*****
***Also I need to give the output load to fill in the LUT

*C1xor xor_out 0 0.365616f
*C2xor xor_out 0 0.791550f
*C3xor xor_out 0 1.583100f
*C4xor xor_out 0 3.166200f
*C5xor xor_out 0 6.332400f
*C6xor xor_out 0 12.664800f
*C7xor xor_out 0 25.329600f

```

```

*****
*****Side Inputs*****
*****As long as in2=0, these inputs do not change*****

*V2xor in2xor 0 DC 0
*V4xor in4xor 0 DC 2.5

*****If in2=1, use these inputs*****

*V2xor in2xor 0 DC 2.5
*V4xor in4xor 0 DC 0

```

ΠΥΛΗ XNOR

Η πύλη XNOR έχει πολλές ομοιότητες με την πύλη XOR:

- Είναι και οι 2 λογικές πύλες, άρα κατασκευάζονται με βάση την συνδιαστική λογική – Δεν έχουν constraint arcs
- Τα συγκεκριμένα μοντέλα προς χαρακτηρισμό αποτελούνται από 2 εισόδους και 1 έξοδο (XNOR2_X1)
- Είναι και οι 2 πύλες Binate – Πρέπει να γνωρίζουμε τα side inputs για να γνωρίζουμε το unateness
- Χαρακτηρίζονται από τα ίδια ακριβώς timing arcs (λόγω των 2 παραπάνω συνθηκών)

Ειδοποιός διαφορά ανάμεσα στις πύλες XOR/XNOR: Η XNOR είναι το αντίθετο της XOR, δηλαδή για bit εισόδου 00 ή 11 η έξοδός της είναι 1 ενώ για bit εισόδου 01 ή 10, η έξοδός της είναι 0.

Όλα τα αρχεία που χρειάζονται για τον χαρακτηρισμό της πύλης θα έχουν ίδια μορφή, όπως φαίνεται παρακάτω

```

57 {
58     "name": "XNOR2_X1",
59     "type": "combinational",
60     "function": "! (A ^ B)",
61     "filepath": "spice_exercises/semester_project/xnor.spice",
62     "input_pins": ["in1xnor", "in2xnor"],
63     "output_pins": ["xnor_out"],
64     "timing_arcs": [
65         {
66             "from_pin": "in1xnor",
67             "to_pin": "xnor_out",
68             "side_input": 0,
69             "unateness": "negative_unate",
70             "lut_template": "Timing_7_7",
71             "cell_rise": "vlsi_measurements/xnor_gate/arcs_from_in1xnor/xnor_cell_rise_si0_lut.json",
72             "cell_fall": "vlsi_measurements/xnor_gate/arcs_from_in1xnor/xnor_cell_fall_si0_lut.json",
73             "rise_transition": "vlsi_measurements/xnor_gate/arcs_from_in1xnor/xnor_rise_transition_si0_lut.json",
74             "fall_transition": "vlsi_measurements/xnor_gate/arcs_from_in1xnor/xnor_fall_transition_si0_lut.json"
75         },
76         {
77             "from_pin": "in2xnor",
78             "to_pin": "xnor_out",
79             "side_input": 0,
80             "unateness": "negative_unate",
81             "lut_template": "Timing_7_7",
82             "cell_rise": "vlsi_measurements/xnor_gate/arcs_from_in2xnor/xnor_cell_rise_si0_lut.json",
83             "cell_fall": "vlsi_measurements/xnor_gate/arcs_from_in2xnor/xnor_cell_fall_si0_lut.json",
84             "rise_transition": "vlsi_measurements/xnor_gate/arcs_from_in2xnor/xnor_rise_transition_si0_lut.json",
85             "fall_transition": "vlsi_measurements/xnor_gate/arcs_from_in2xnor/xnor_fall_transition_si0_lut.json"
86         }
87     ],
88 }

```

Ακόμη και τα LUTs (στιγμιότυπο από την πρώτη στήλη του fall transition με side input 0, στον φάκελο arcs_from_in1xnor)

```

1 {
2     "measurements": [
3         {
4             "input_slew": 0.00231025,
5             "output_load": [
6                 {
7                     "output_load": 0.365616,
8                     "value": 0.686521
9                 },
10                {
11                    "output_load": 0.813006,
12                    "value": 0.687327
13                },
14                {
15                    "output_load": 1.626010,
16                    "value": 0.688791
17                },
18                {
19                    "output_load": 3.252020,
20                    "value": 0.691720
21                },
22                {
23                    "output_load": 6.504050,
24                    "value": 0.697576
25                },
26                {
27                    "output_load": 13.008100,
28                    "value": 0.709290
29                },
30                {
31                    "output_load": 26.016200,
32                    "value": 0.732717
33                }
34            ]
35        }
36    ]
37 }

```

Οι τιμές που παρέχονται από το μοντέλο Timing_7_7 της βιβλιοθήκης διαφέρουν λίγο στο output load σε σχέση με την προηγούμενη πύλη

```
rise_transition(Timing_7_7) {  
    index_1 ("0.00231025,0.0112628,0.0426883,0.102700,0.196195,0.327379,0.500000");  
    index_2 ("0.365616,0.813006,1.626010,3.252020,6.504050,13.008100,26.016200");  
}
```

Τα αρχεία spice που χρησιμοποίησα για να χαρακτηρίσω όλα τα timing arcs της πύλης είναι τα:

- xnor_inputs_rising_in1.spice
- xnor_inputs_rising_in2.spice
- xnor_inputs_falling_in1.spice
- xnor_inputs_falling_in2.spice

Τα αρχεία λειτουργούν ακριβώς με τον ίδιο τρόπο που λειτουργούν και τα αντίστοιχα αρχεία της πύλης XOR. Πάλι υπάρχουν αναλυτικές οδηγίες στα σχόλια του κάθε αρχείου και θέλει μεγάλη προσοχή να επιλεγεί μία μόνο τιμή από κάθε παράμετρο.

D FLIP FLOP

Το D Flip Flop, είναι ένα ακολουθιακό κύκλωμα, δηλαδή έχει μνήμη, σε αντίθεση με τις πύλες XOR/XNOR όπου είναι συνδιαστικά κυκλώματα. Οπότε ο τρόπος προσέγγισης του χαρακτηρισμού είναι λίγο διαφορετικός. Αποτελείται από 2 εισόδους, την είσοδο δεδομένων D και το ρολόι Clk, και από 2 εξόδους, την Q και την συμπληρωματική της nQ. Το Flip Flop που υλοποίησα στο προηγούμενο μέρος της εργασίας άλλαζε κατάσταση μόνο στην θετική ακμή του ρολογιού, άρα είναι posedge. Η λειτουργία του είναι: Σε κάθε θετική ακμή του ρολογιού, η έξοδος Q του ρολογιού παίρνει την τιμή που έχει η είσοδος δεδομένων D εκείνη την χρονική στιγμή (αρκεί η είσοδος D να μην έχει αλλάξει λογική κατάσταση πολύ κοντά με το ρολόι).

Εύλογα λοιπόν, μπορούμε να συμπεράνουμε ότι προκύπτουν 8 timing arcs, 4 για κάθε έξοδο, 2 cell delays, ένα όταν D=0 και άλλο ένα όταν D=1 (προφανώς, αφού το κύκλωμα είναι posedge, η μετάβαση του ρολογιού θα είναι μόνο από 0 σε 1) και άλλα 2 output transitions. Όμως, επειδή το DFF είναι ακολουθιακό κύκλωμα, θα έχει και χρονικούς περιορισμούς (constraint arcs), δηλαδή μερικοί περιορισμοί ανάμεσα στην είσοδο D και την είσοδο του ρολογιού που εάν παραβιαστούν, το κύκλωμα θα έχει ακανόνιστη, άρα και μη επιθυμητή, συμπεριφορά. Στο DFF έχουμε 2 είδη τέτοιων περιορισμών: το setup_rising και το hold_rising. Το setup_rising αφορά το πόσο νωρίτερα από το ρολόι πρέπει να φτάσει η είσοδος D στο κύκλωμα, ενώ το hold_rising από ποιο χρονικό διάστημα και μετά το ρολόι, μπορεί η είσοδος D να αλλάξει λογική κατάσταση. Με πιο απλά λόγια, το setup_rising και το hold_rising δηλώνουν για πόσο χρόνο πριν και μετά από την θετική ακμή του ρολογιού, αντίστοιχα, πρέπει η είσοδος D να παραμείνει σταθερή. Προκύπτουν επομένως ακόμη 4 arcs, αυτήν την φορά όμως constraint arcs, 2 setup_rising, 1 όταν η έξοδος «ανεβαίνει» και άλλο 1 όταν «κατεβαίνει», και άλλα 2 hold_rising. Προφανώς, από την στιγμή που το DFF είναι posedge, δεν έχω μεταβολή εξόδου στις αρνητικές ακμές του ρολογιού, άρα ούτε setup_falling και hold_falling.

Η μέτρηση των timing arcs γίνεται και εδώ πέρα με τον ίδιο ακριβώς τρόπο στο NGspice. Όμως, το NGspice δεν μας παρέχει την δυνατότητα να μετρήσουμε απευθείας τα constraint arcs, οπότε καταφεύγουμε στην μέθοδο trial and error. Αυτό που έκανα σε αυτό το βήμα ήταν να πλησιάζω συνεχώς την είσοδο D στο ρολόι πολύ κοντά έως ότου αυτό να παύει να λειτουργεί σωστά (κρατώντας σταθερά τα

slew των 2 εισόδων σε κάθε μέτρηση, και την χρονική στιγμή του παλμού του ρολογιού). Όταν βρω αυτόν τον χρόνο (με ακρίβεια 3 δεκαδικών), αφήνω και 5% περιθώριο λάθους στον χρόνο που βρήκα για να είμαι εντελώς σίγουρος ότι δεν θα υπάρξει καμία παραβίαση των setup time και hold time.

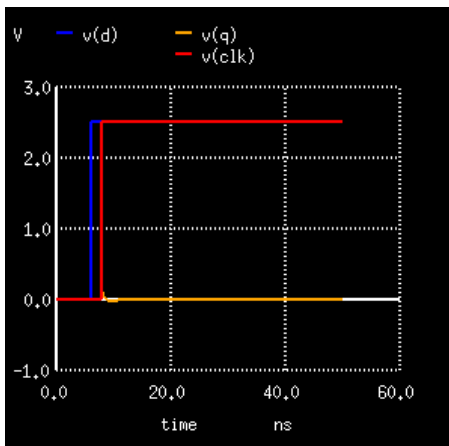
Τα constraint arcs μπορούμε και αυτά να τα δούμε ως συναρτήσεις 2 μεταβλητών με ανεξάρτητες μεταβλητές τα constraint pin transition και related pin transition, δηλαδή τα input slew της εισόδου D και του ρολογιού αντίστοιχα. Οπότε και αυτά θα περιγράφονται από LUTs.

Τα templates της βιβλιοθήκης που θα χρησιμοποιήσω αυτήν την φορά και περιέχουν τα input slew των εισόδων φαίνονται παρακάτω

<pre>rise_constraint(Setup_3_3) { index_1 ("0.00231025,0.112657,0.500000"); index_2 ("0.00231025,0.112657,0.500000");</pre>	<pre>fall_constraint(Hold_3_3) { index_1 ("0.00231025,0.112657,0.500000"); index_2 ("0.00231025,0.112657,0.500000");</pre>
---	--

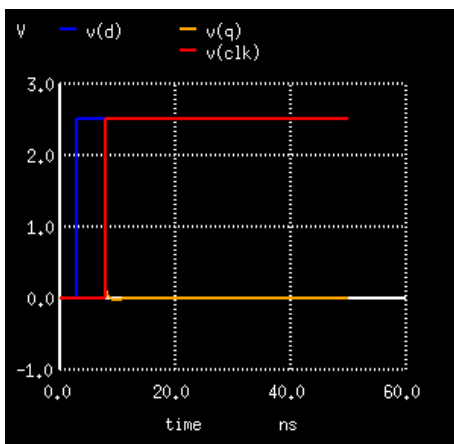
Παρακάτω φαίνεται το παράδειγμα για related_pin_transition = constraint_pin_transition = 0.00231025 και για ανοδική έξοδο (rise constraint). Ξεκινώντας από απόσταση 2ns, έχουμε:

2ns



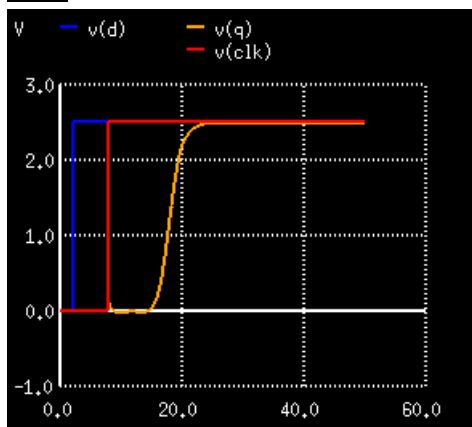
Προφανώς, το περιθώριο μεταξύ των παλμών είναι υπερβολικά μικρό και πρέπει να το αυξήσω κι άλλο

5ns



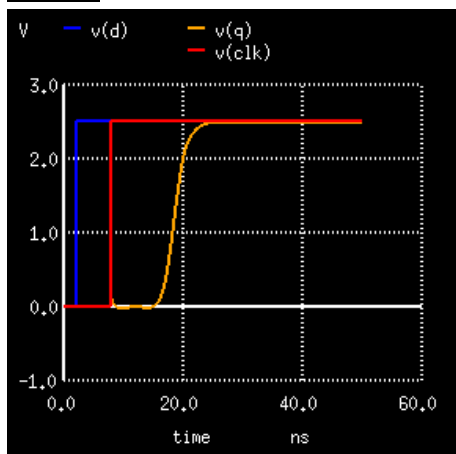
Ακόμη το περιθώριο παραμένει πολύ μικρό, αλλά πλησιάζουμε

6ns



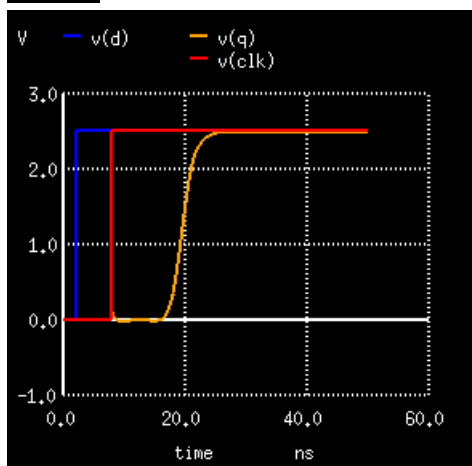
Τώρα έχουμε έξοδο, οπότε το setup time είναι κάπου ενδιάμεσα στα 5ns και τα 6ns

5.9ns



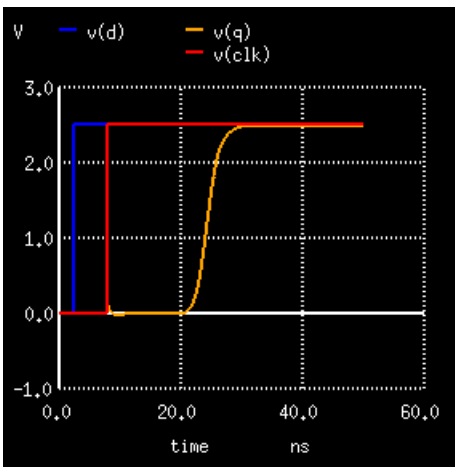
Ακόμη έχουμε έξοδο, μειώνουμε κι άλλο την απόσταση των εισόδων

5.8ns



Ακόμη τα ίδια, μειώνουμε κι άλλο την απόσταση

5.7ns



Πρέπει να μειώσουμε κι άλλο την απόσταση

Σημείωση: Εκτός από το setup time, μετρώ 3 ακόμη χρόνους κάθε φορά που τρέχω το NGspice:

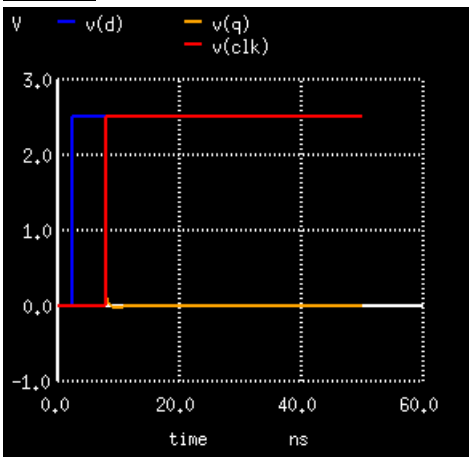
```
setup_time      = 5.700000e-09 targ= 7.997112e-09 trig= 2.297112e-09
cell_delay      = 1.641369e-08 targ= 2.441080e-08 trig= 7.997112e-09
rising_slew_d   = 2.310250e-12 targ= 2.298267e-09 trig= 2.295957e-09
rising_slew_clk = 2.310250e-12 targ= 7.998267e-09 trig= 7.995957e-09
```

Το cell delay δεν έχει κάποια πρακτική σημασία, απλά βλέπω την καθυστέρηση σε κάθε βήμα. Έχει όμως ενδιαφέρον να παρατηρήσουμε το πόσο πολύ έχει αυξηθεί το cell delay, δηλαδή πόσο πιο αργό έχει γίνει το κύκλωμα, όταν έχουμε πλησιάσει την είσοδο D και το ρολόι τόσο κοντά (είναι περίπου 16ns ενώ στην μέτρηση των timing arcs δεν έχει ξεπεράσει τα 8ns μέχρι στιγμής).

Από την άλλη μεριά, οι 2 τελευταίες μετρήσεις με βοηθάνε πάρα πολύ σε κάθε βήμα να βεβαιωθώ ότι έχω δώσει το σωστό input slew στο κύκλωμα, ειδικά το rising_slew_d, καθώς σε κάθε μέτρηση μεταβάλλω την εντολή εισόδου D, οπότε είναι πολύ εύκολο να ξεφύγει κάποιο λάθος

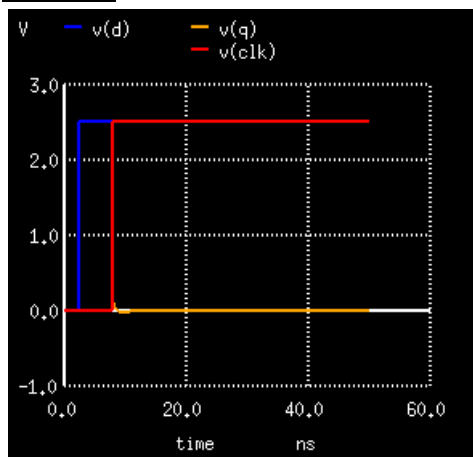
Συνεχίζω με τις μετρήσεις από την τιμή 5.6ns

5.6ns



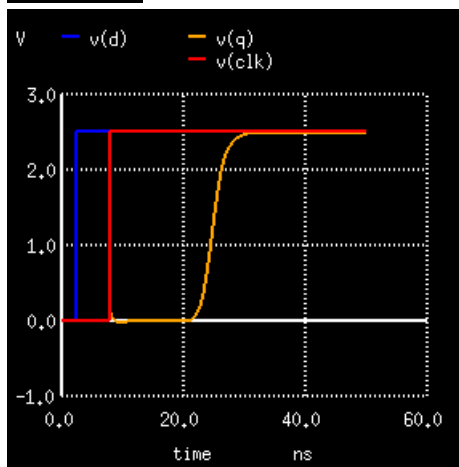
Παρατηρώ ότι η έξοδος του κυκλώματος πλέον έχει χαλάσει, άρα το setup time βρίσκεται μεταξύ 5.6ns και 5.7ns

5.69ns



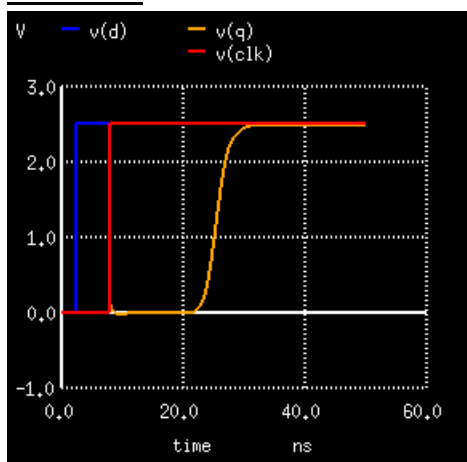
Η έξοδος ακόμα δεν είναι σωστή, επομένως το setup time βρίσκεται μεταξύ 5.69ns και 5.70ns

5.699ns



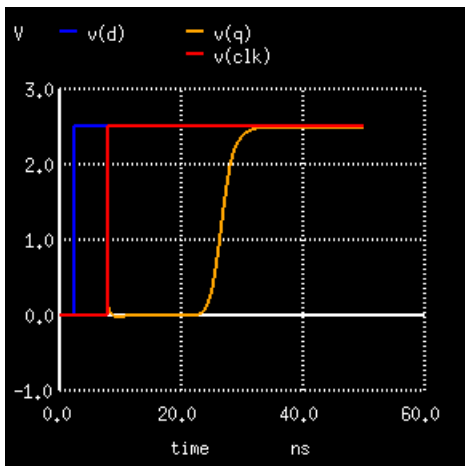
Έχουμε έξοδο, πλησιάζω κι άλλο την είσοδο D στο ρολόι

5.698ns



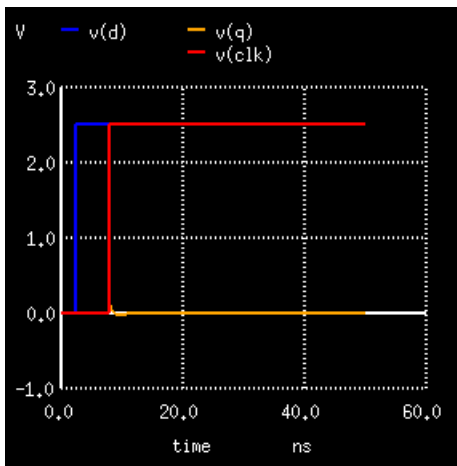
Μπορώ να πλησιάσω κι άλλο τις εισόδους

5.697ns



Έχω κι άλλο περιθώριο

5.696ns



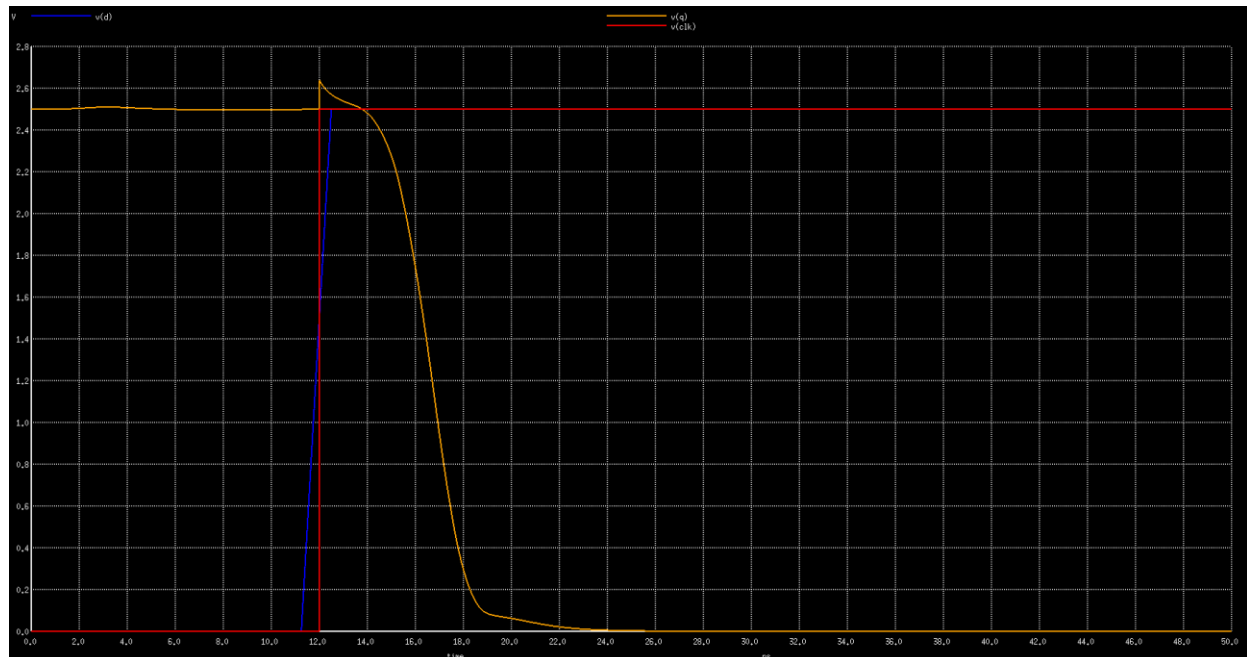
Η έξοδος έχει χαλάσει

Από την στιγμή που υπολόγισα τον χρόνο με ακρίβεια 3 δεκαδικών ψηφίων σταματάω και δέχομαι ως τελική μέτρηση, την τελευταία μέτρηση στην οποία το κύκλωμα λειτουργεί σωστά, δηλαδή τα 5.697ns. Όμως, είναι πολύ σημαντικό να αφήσω και ένα περιθώριο λάθους για αποφυγή προβλημάτων οριακών περιπτώσεων. Έστω ότι το περιθώριο λάθους είναι 5%. Τότε η τελική τιμή του setup time σε αυτήν την περίπτωση θα είναι: $5.697\text{ns} * 1.05 = 5.981850\text{ns}$

Με την ίδια τεχνική υπολογίζονται και οι υπόλοιποι χρόνοι που χρειάζονται για την συμπλήρωση του LUT.

Η παραπάνω τεχνική, ενώ είναι σωστή σαν σκέψη και θα έχω το επιθυμητό αποτέλεσμα, είναι αρκετά χρονοβόρα κατά μέσο όρο (άσχετα που σε αυτήν την περίπτωση προέκυψαν αρκετά γρήγορα οι μετρήσεις). Μια καλύτερη τεχνική θα ήταν κάθε φορά που βρίσκω ένα διάστημα τιμών, αντί να ψάχνω στο διάστημα ξεκινώντας από το ένα άκρο και να καταλήξω στο άλλο, να βρω την τιμή στην μέση του διαστήματος, και αναλόγως με το αποτέλεσμα, να συνεχίσω να ψάχνω και στο αντίστοιχο, μισό σε σχέση με το αρχικό, διάστημα (να κάνω δηλαδή binary search αντί για linear search). Έτσι, εάν έχω ένα διάστημα από 5.69 έως 5.70 για παράδειγμα (με 10 πιθανές τιμές που να περιλαμβάνουν 3 δεκαδικά ψηφία), θα κάνω κατά μέσο όρο $\log_2(10) = 3.322$ μετρήσεις για να βρω το επόμενο δεκαδικό ψηφίο του constraint arc, ενώ με τον τρόπο που περιέγραψα επάνω θέλω, κατά μέσο όρο $10/2 = 5$ μετρήσεις (10 μετρήσεις worst case).

Για την εύρεση του hold time, ακολουθώ την ίδια τεχνική, αλλά αυτήν την φορά «σέρνω» την είσοδο δεδομένων από τα δεξιά προς τα αριστερά (πλησιάζω δηλαδή τον παλμό του ρολογιού από τα δεξιά). Όμως, σε όλες τις περιπτώσεις δεν παρατηρώ λανθασμένη λειτουργία του κυκλώματος, ακόμη και όταν η είσοδος δεδομένων εφάπτεται με τον παλμό του ρολογιού. Παρακάτω φαίνεται αυτό το πρόβλημα, έχοντας θέσει ως input slew στο ρολόι 0.5ns και στην είσοδο D 0.00231025



Έτσι, καταλήγω στο συμπέρασμα ότι το hold time είναι αρνητικό. Όμως, επειδή το αρνητικό hold time δεν έχει φυσική σημασία στο κύκλωμα και επειδή σε κάθε μέτρηση αφήνω και ένα περιθώριο λάθους, έθεσα ως hold time την τιμή 0.001ns σε όλες περιπτώσεις είχα αρνητικό hold time.

Κάτι που ξέχασα να αναφέρω, είναι ότι το DFF, επειδή είναι ακολουθιακό κύκλωμα, θα πρέπει να περιέχει μέσα στο αντικείμενο που το χαρακτηρίζει, έναν πίνακα που ονομάζεται ff group. Ουσιαστικά, αυτός ο πίνακας υποδηλώνει την λειτουργικότητα των pins στο standard cell. Οπότε, το index του DFF στον πίνακα models θα έχει την εξής μορφή

```

111 {
112     "name": "DFF_X1",
113     "type": "sequential",
114     "ff_group": [
115         {
116             "ff_type": "synchronous",
117             "next_state_output": "IQ",
118             "not_next_state_output": "nIQ",
119             "data_input": "D",
120             "clock_input": "clk"
121         }
122     ],
123     "filepath": "spice_exercises/semester_project/dff.spice",
124     "input_pins": ["D", "clk"],
125     "output_pins": ["Q", "nQ"],
126     "timing_arcs": [

```

Όπως φαίνεται και στην εικόνα, το ff group μας λέει εάν το κύκλωμα είναι σύγχρονο ή ασύγχρονο και το ρόλο του κάθε pin στο κύκλωμα

Όσο αφορά τα timing arcs, η μόνη διαφορά σε σχέση με τις πύλες XOR/XNOR είναι ότι αντί να έχω 16 timing arcs, έχω 4 (για κάθε pin εξόδου), καθώς η μόνη είσοδος η οποία προκαλεί αλλαγή στην έξοδο είναι το ρολόι, και η αλλαγή έρχεται μόνο στην θετική ακμή του ρολογιού, οπότε το πεδίο των timing arcs στο config file θα μοιάζει κάπως έτσι. Σε αυτήν την περίπτωση όμως, για να ονομάσω κατάλληλα ένα αρχείο, πρέπει στο όνομά του να αναφέρω και το pin εξόδου το οποίο σχετίζεται με αυτό το timing arc

```
126 "timing_arcs": [  
127 {  
128     "from_pin": "clk",  
129     "to_pin": "Q",  
130     "side_input": 1,  
131     "unateness": "non_unate",  
132     "lut_template": "Timing_7_7",  
133     "cell_rise": "vlsi_measurements/dff_circuit/timing_arcs/dff_cell_rise_Q_exit_lut.json",  
134     "cell_fall": "vlsi_measurements/dff_circuit/timing_arcs/dff_cell_fall_Q_exit_lut.json",  
135     "rise_transition": "vlsi_measurements/dff_circuit/timing_arcs/dff_rise_transition_Q_exit_lut.json",  
136     "fall_transition": "vlsi_measurements/dff_circuit/timing_arcs/dff_fall_transition_Q_exit_lut.json"  
137 },  
138 {  
139     "from_pin": "clk",  
140     "to_pin": "nQ",  
141     "side_input": 1,  
142     "unateness": "non_unate",  
143     "lut_template": "Timing_7_7",  
144     "cell_rise": "vlsi_measurements/dff_circuit/timing_arcs/dff_cell_rise_nQ_exit_lut.json",  
145     "cell_fall": "vlsi_measurements/dff_circuit/timing_arcs/dff_cell_fall_nQ_exit_lut.json",  
146     "rise_transition": "vlsi_measurements/dff_circuit/timing_arcs/dff_rise_transition_nQ_exit_lut.json",  
147     "fall_transition": "vlsi_measurements/dff_circuit/timing_arcs/dff_fall_transition_nQ_exit_lut.json"  
148 }  
149 ],
```

Τέλος, όπως ήδη έχω αναφέρει, επειδή το κύκλωμα είναι ακολουθιακό, θα έχει και constraint arcs, οπότε παρακάτω φαίνεται και ο πίνακας των constraint arcs που δημιουργήσα μέσα στο index που περιλαμβάνει το DFF.

```
150 "constraint_arcs": [  
151 {  
152     "from_pin": "D",  
153     "to_pin": "clk",  
154     "constraint_type": "setup_rising",  
155     "lut_template": "Setup_3_3",  
156     "rise_constraint": "vlsi_measurements/dff_circuit/constraint_arcs/dff_setup_rising_rise_constraint_lut.json",  
157     "fall_constraint": "vlsi_measurements/dff_circuit/constraint_arcs/dff_setup_rising_fall_constraint_lut.json"  
158 },  
159 {  
160     "from_pin": "D",  
161     "to_pin": "clk",  
162     "constraint_type": "hold_rising",  
163     "lut_template": "Hold_3_3",  
164     "rise_constraint": "vlsi_measurements/dff_circuit/constraint_arcs/dff_hold_rising_rise_constraint_lut.json",  
165     "fall_constraint": "vlsi_measurements/dff_circuit/constraint_arcs/dff_hold_rising_fall_constraint_lut.json"  
166 }  
167 ],  
168
```

Παρακάτω φαίνεται και μια στήλη από το αρχείο dff_setup_rising_fall_constraint_lut.json. Το αρχείο μοιάζει αρκετά με ένα αρχείο που περιγράφει timing arcs, με μόνες διαφορές να είναι τα ονόματα των indexes (constrained_pin_transition και related_pin_transition αντί για input_slew και output_load) και το ότι τώρα χρησιμοποιούμε διαφορετικό LUT template (Setup_3_3 αντί για Timing_7_7, φαίνεται και στο αντίστοιχο πεδίο lut_template μέσα στο config file).

```

1  {
2      "measurements": [
3          {
4              "constrained_pin_transition": 0.00231025,
5              "related_pin_transition": [
6                  {
7                      "related_pin_transition": 0.00231025,
8                      "value": 7.827750
9                  },
10                 {
11                     "related_pin_transition": 0.112657,
12                     "value": 7.805770
13                 },
14                 {
15                     "related_pin_transition": 0.5,
16                     "value": 7.754132
17                 }
18             ]
19         },

```

ΔΗΜΙΟΥΡΓΙΑ ΡΟΗΣ ΧΑΡΑΚΤΗΡΙΣΜΟΥ

Αφού χαρακτήρισα όλα τα παραπάνω timing και constraint arcs με την χρήση LUTs σε ένα configuration file, ήρθε η ώρα να δημιουργήσω ένα εργαλείο το οποίο θα δέχεται ως είσοδο το configuration file και θα εξάγει ένα .lib file (την παραγόμενη βιβλιοθήκη). Το εργαλείο θα γραφτεί σε γλώσσα python καθώς η python μπορεί πολύ εύκολα να διαβάσει αρχεία JSON (με την χρήση της βιβλιοθήκης JSON), και γενικά, είναι αρκετά εύκολη στην χρήση.

Αρχικά, το παραγόμενο lib file θα πρέπει να περιέχει το Library Header, δηλαδή γενικές πληροφορίες για την βιβλιοθήκη (όνομα, έκδοση, τεχνολογία, default μονάδες μέτρησης κτλ). Το κείμενο βρίσκεται στο αρχείο library_header.py, διαβάζεται με την εντολή open στο read mode και γράφουμε το περιεχόμενό του στο my_library.lib, την τελική βιβλιοθήκη, πάλι με την εντολή open, αλλά σε write mode.

```

1  * *****
2  * *
3  * *      My Library for the 2nd part of the semester project in ECE327      *
4  * * Designed according to the Cadence's NangateOpenCellLibrary_slow.txt Library *
5  * *      Petroglou Spyridon 03185                                           *
6  * *
7  * *****
8  *
9  * Temperature          : 27C
10 * Voltage              : 2.5V
11 *
12
13 library (NangateOpenCellLibrary) {
14
15     /* Documentation Attributes */
16     date                : "Tue 10 Oct 2023, 20:27:16";
17     revision             : "revision 1.0";
18     comment              : "Copyright (c) 2004-2011 Nangate Inc. All Rights Reserved.";
19
20     /* General Attributes */
21     technology           (cmos);
22     delay_model           : table_lookup;
23
24     /* Units Attributes */
25     time_unit             : "1ns";
26     leakage_power_unit    : "1nW";
27     voltage_unit          : "1V";
28     current_unit          : "1mA";

```

Η αρχή της βιβλιοθήκης μου. Το μόνο που μεταβάλλεται στον παραπάνω κώδικα είναι το πεδίο date, το οποίο ανανεώνεται κάθε φορά που το αρχείο τρέχει.

Αμέσως μετά γίνονται οι αρχικοποιήσεις των lut templates. Τα templates που πρέπει να τυπώσω παρέχονται από την Nangate βιβλιοθήκη και φαίνονται στο στιγμιότυπο στην σελίδα 4. Οπότε τα αποθηκεύω σε ένα νέο αρχείο, το οποίο το ονομάζω lut_templates.txt και με τις κατάλληλες ανακατευθύνσεις αρχείων, γράφω το περιεχόμενό του στο σημείο που τελειώνει το library header. Πρακτικά, θα μπορούσα να χρησιμοποιήσω το ίδιο αρχείο και για το library header, και για τα lut templates, αλλά έτσι επιτυγχάνω την καλύτερη οργάνωση των δεδομένων μου. Οπότε το lib file θα μοιάζει κάπως έτσι σε αυτό το σημείο

```
41
42  /* Library Look_Up Tables Templates */
43
44  lu_table_template (Hold_3_3) {
45    variable_1 : constrained_pin_transition;
46    variable_2 : related_pin_transition;
47    index_1 ("0.0010,0.0020,0.0030");
48    index_2 ("0.0010,0.0020,0.0030");
49  }
50
51  lu_table_template (Recovery_3_3) {
52    variable_1 : constrained_pin_transition;
53    variable_2 : related_pin_transition;
54    index_1 ("0.0010,0.0020,0.0030");
55    index_2 ("0.0010,0.0020,0.0030");
56  }
57
```

Τώρα ήρθε η ώρα να αξιοποιήσω το configuration file για να ολοκληρώσω την βιβλιοθήκη μου. Το επόμενο πεδίο που συναντάμε είναι το cell rules, το οποίο περιέχει το όνομα του standard cell, διάφορες άλλες πληροφορίες για το cell και το επόμενο πεδίο της βιβλιοθήκης, τα pins. Δημιουργώντας ένα python loop που διαβάζει όλα τα standard cells που δημιούργησα και χαρακτήρισα, προκύπτει

```
81  cell("XOR2_X1") {
82
83  }
84  cell("XNOR2_X1") {
85
86  }
87  cell("DFF_X1") {
88
89  }
```

Αν το cell είναι sequential, τότε το πεδίο cell θα πρέπει να περιέχει και ένα ff group. Το ff group περιγράφει ποιο είναι το ρολόι του κυκλώματος, ποια είναι η είσοδος δεδομένων και, εάν το ff είναι ασύγχρονο, τα preset και clear pins. Το config file περιέχει όλες τις παραπάνω πληροφορίες μέσα σε ένα αντικείμενο ενός πίνακα που ονομάζεται ff_group. Η πρώτη θέση αυτού του πίνακα δηλώνει εάν το flip flop είναι σύγχρονο ή ασύγχρονο, ενώ οι υπόλοιπες θέσεις περιγράφουν την λειτουργία του pin. Οπότε η βιβλιοθήκη τώρα θα μοιάζει κάπως έτσι

```

105     cell("DFF_X1") {
106         ff ("IQ","nIQ") {
107             next_state: "D"
108             clocked_on: "Clk"
109         }

```

Με τον ίδιο τρόπο, δημιουργώ άλλα δύο loops, ένα για τα input pins και ένα για τα output pins, για το πεδίο pin. Και η βιβλιοθήκη μου θα φαίνεται κάπως έτσι μετά από αυτό το βήμα

```

93     cell("XNOR2_X1") {
94
95         pin("in1xnor") {
96
97         }
98         pin("in2xnor") {
99
100        }
101        pin("xnor_out") {
102
103        }
104    }
105    cell("DFF_X1") {
106        ff ("IQ","nIQ") {
107            next_state: "D"
108            clocked_on: "Clk"
109        }
110        pin("D") {
111
112        }
113        pin("Clk") {
114
115        }

```

Τα pin rules περιέχουν χρήσιμες πληροφορίες για το εκάστοτε pin. Τα input pins περιέχουν την κατεύθυνση του pin (εάν το pin είναι input ή output, εδώ προφανώς το direction θα είναι input) και την χωρητικότητα του pin. Τα output pins περιέχουν την κατεύθυνση του pin και την Boolean συνάρτηση του κυκλώματος. Τέλος, τα pins περιέχουν και τα κατάλληλα arcs μέσα σε ένα πεδίο timing(). Τα input pins των ακολουθιακών κυκλωμάτων περιέχουν τα constraint arcs (setup_rising και hold rising όλα τα ακολουθιακά κυκλώματα, επιπλέον recovery_rising και removal_rising τα ασύγχρονα κυκλώματα)

```

96     cell("XNOR2_X1") {
97
98         pin("in1xnor") {
99             direction: input
100             capacitance: 15.4
101         }
102         pin("in2xnor") {
103             direction: input
104             capacitance: 15.4
105         }
106         pin("xnor_out") {
107             direction: output
108             function: !(A ^ B)
109         }
110     }
111     cell("DFF_X1") {
112         ff ("IQ","nIQ") {
113             next_state: "D"
114             clocked_on: "Clk"
115         }
116         pin("D") {
117             direction: input

```

Με την προσθήκη μερικών πεδίων στο pin, η βιβλιοθήκη θα έχει την παραπάνω μορφή. Παρατηρούμε ότι όσο προχωράει το στήσιμο της βιβλιοθήκης, τόσο πιο κοντά τείνει να μοιάσει στην έτοιμη nandgate βιβλιοθήκη. Οπότε, προχωράμε στο επόμενο βήμα, το οποίο είναι η προσθήκη των arcs, των οποίων τα LUTs υπάρχουν αναλυτικά μέσα στο config file. Τα arcs πρέπει να τυπώνονται μέσα στο πεδίο timing(), το οποίο αρχικά περιέχει τις εξής πληροφορίες: related pin, μας δείχνει από ποιο pin «ξεκινάει» το arc, timing type, που μας δείχνει ποιον τύπο constraint arc έχω (προφανώς μόνο για ακολουθιακά κυκλώματα) και το timing sence, που μας δείχνει το unateness του pin.

```

91         pin("xor_out") {
92             direction: output
93             function: (A ^ B)
94
95             timing() {
96
97                 "related_pin": in1xor
98                 "timing_sence": positive_unate
99
100         cell_rise (Timing_7_7) {
101             index_1 ("0.00231025", "0.0112628", "0.0426883", "0.1027", "0.196195", "0.327379", "0.5")
102             index_2 ("0.365616", "0.79155", "1.5831", "3.1662", "6.3324", "12.6648", "25.3296")
103         }
104
105         cell_fall (Timing_7_7) {
106             index_1 ("0.00231025", "0.0112628", "0.0426883", "0.1027", "0.196195", "0.327379", "0.5")
107             index_2 ("0.365616", "0.79155", "1.5831", "3.1662", "6.3324", "12.6648", "25.3296")
108         }
109
110         rise_transition (Timing_7_7) {
111             index_1 ("0.00231025", "0.0112628", "0.0426883", "0.1027", "0.196195", "0.327379", "0.5")
112             index_2 ("0.365616", "0.79155", "1.5831", "3.1662", "6.3324", "12.6648", "25.3296")
113         }
114
115         fall_transition (Timing_7_7) {
116             index_1 ("0.00231025", "0.0112628", "0.0426883", "0.1027", "0.196195", "0.327379", "0.5")
117             index_2 ("0.365616", "0.79155", "1.5831", "3.1662", "6.3324", "12.6648", "25.3296")
118         }
119     }

```

Αφού τυπώσω και τα indexes, πρέπει μετά να τυπώσω το LUT των arcs.

```

400     timing() {
401
402         "related_pin": in2xnor;
403         "timing_sence": negative_unate;
404
405         cell_rise (Timing_7_7) {
406             index_1 ("0.00231025", 0.0112628, 0.0426883, 0.1027, 0.196195, 0.327379, 0.5);
407             index_2 ("0.365616", 0.813006, 1.62601, 3.25202, 6.50405, 13.0081, 26.0162);
408             values ("4.297607, 4.292658, 4.276639, 4.268947, 4.285291, 4.32913, 4.399889", /
409                 "4.299512, 4.294564, 4.278546, 4.270857, 4.287204, 4.331051, 4.401817", /
410                 "4.302973, 4.298026, 4.282012, 4.274327, 4.290682, 4.33454, 4.405321", /
411                 "4.309894, 4.304949, 4.288942, 4.281265, 4.297635, 4.341517, 4.412327", /
412                 "4.323729, 4.318788, 4.302795, 4.295136, 4.311535, 4.355464, 4.426331", /
413                 "4.351376, 4.346443, 4.330476, 4.322851, 4.339306, 4.383328, 4.454303", /
414                 "4.406572, 4.401657, 4.385741, 4.37818, 4.394743, 4.438941, 4.510126");
415         }
416
417         cell_fall (Timing_7_7) {
418             index_1 ("0.00231025", 0.0112628, 0.0426883, 0.1027, 0.196195, 0.327379, 0.5);
419             index_2 ("0.365616", 0.813006, 1.62601, 3.25202, 6.50405, 13.0081, 26.0162);
420             values ("1.885337, 2.170491, 2.1607, 2.177139, 2.193962, 2.228088, 2.27338", /
421                 "1.886434, 2.17159, 2.161798, 2.178239, 2.195087, 2.22919, 2.274482", /
422                 "1.888428, 2.173587, 2.163794, 2.180237, 2.197136, 2.231192, 2.276485", /
423                 "1.892414, 2.177579, 2.167785, 2.184232, 2.201214, 2.235194, 2.28049", /
424                 "1.900381, 2.185559, 2.17576, 2.192216, 2.209566, 2.243192, 2.28849", /
425                 "1.916294, 2.201497, 2.191691, 2.208161, 2.226014, 2.259155, 2.304462", /
426                 "1.947984, 2.23329, 2.223468, 2.23996, 2.258796, 2.290978, 2.336308");
427         }
428     }

```

Και τα constraint arcs κατασκευάζονται και τυπώνονται με τον ίδιο ακριβώς τρόπο, όπως φαίνεται και από τις παρακάτω εικόνες

```
580 pin("D") {
581     direction: input;
582     capacitance: 15.4;
583
584     timing() {
585
586         "related_pin": Clk;
587         "timing_type": setup_rising";
588     }
589 pin("Clk") {
590     direction: input;
591     capacitance: 15.4;
592 }
```

D FLIP FLOP ME PRESET ΚΑΙ CLEAR

Τέλος, στα πλαίσια της εργασίας πρέπει να χαρακτηρίσω και ένα ασύγχρονο DFF με preset και clear. Η λειτουργία του μοιάζει πολύ με αυτήν του σύγχρονου DFF. Η δομή του είναι ακριβώς η ίδια, αλλά περιέχει επιπλέον δύο input pins, το preset και το clear. Η διαφορά του με το DFF είναι ότι εάν το Preset έχει την τιμή 1 και το Clear την τιμή 0, η έξοδος Q γίνεται 1 (και η συμπληρωματική της 0), ενώ εάν το Preset είναι 0 και το Clear 1, τότε η έξοδος Q γίνεται 0 και η συμπληρωματική της 1. Εάν και τα 2 παραπάνω pin είναι 0, τότε το κύκλωμα λειτουργεί σαν ένα σύγχρονο DFF (δηλαδή η έξοδος εξαρτάται μόνο από το ρολόι), ενώ απαγορεύεται οι 2 αυτές είσοδοι να είναι ταυτόχρονα 1 (invalid κατάσταση)

Το DFFRS το άφησα για το τέλος γιατί ήταν πολύ πιο περίπλοκο από τα υπόλοιπα κυκλώματα (περιλαμβάνει 8 constraint arcs και 72 timing arcs). Η μέτρηση όλων αυτών των arcs αποτελεί μια χρονοβόρα διαδικασία που δεν έχω καταφέρει να την φέρω εις πέρας ακόμα. Όμως, αναγράφω ποια constraint arcs και ποια timing arcs πρέπει να μετρηθούν για να ολοκληρωθεί και αυτό το βήμα.

- Έχω 2 setup_rising και 2 hold_rising τα οποία προκύπτουν θέτοντας 0 στις ασύγχρονες εισόδους (κάνοντας δηλαδή το κύκλωμά μου να λειτουργεί σαν ένα DFF)
- Έχω 1 recovery_rising και 1 removal_rising από το Preset προς το ρολόι, όταν το Clear είναι 0. Υποδηλώνουν τον χρόνο που δεν πρέπει να ενεργοποιηθεί το Preset πριν και μετά τον παλμό του ρολογιού αντίστοιχα (προφανώς μιλάμε μόνο για rise_constraint από την στιγμή που μιλάμε μόνο για ενεργοποίηση του Preset)
- Έχω 1 recovery_rising και 1 removal_rising από το Clear προς το ρολόι, όταν το Preset είναι 0. Μετριέται με τον ίδιο ακριβώς τρόπο όπως και τα παραπάνω

Τα παραπάνω αφορούν την μέτρηση των constraint arcs. Για τα timing arcs θα ισχύει

- Έχω 4 timing arcs (την κλασσική τετράδα cell_rise, cell_fall, rise_transition και fall_transition) από το ρολόι προς την έξοδο Q, όταν οι ασύγχρονες έξοδοι έχουν την τιμή 0
- Από την είσοδο Clear προς την έξοδο Q, έχω περισσότερα timing arcs. Κανονικά, θα έπρεπε να είχα τόσες τετράδες, όσες και οι πιθανοί συνδιασμοί των πλευρικών εισόδων (3 πλευρικές είσοδοι =>

$2^3 = 8$ πιθανοί συνδιασμοί), δηλαδή 32 τετράδες. Όμως, στις περιπτώσεις που η είσοδος Preset είναι 1, για να αποφύγω την invalid κατάσταση, θα πρέπει η είσοδος Clear να πηγαίνει μόνο από το 1 στο 0 και όχι από το 0 στο 1, οπότε αυτό θα δημιουργήσει μόνο cell_fall και fall_transition (η είσοδος clear «κατεβαίνει», η έξοδος «κατεβαίνει» => positive unate). Αυτό σημαίνει ότι όταν Preset = 1, δηλαδή 4 φορές, θα έχω 2 timing arcs λιγότερα στην έξοδο Q. Άρα συνολικά θα έχω $32 - (2 \cdot 4) = 32 - 8 = 24$ timing arcs

- Τα υπόλοιπα 8 timing arcs τοποθετούνται από την είσοδο Preset, όταν η είσοδος Clear είναι 1. Με την ίδια λογική, έχω 4 δυάδες από cell_rise και rise_transition, κάθε φορά που η είσοδος Preset «κατεβαίνει»

Μέχρι στιγμής έχω 36 timing arcs. Με την ίδια λογική έχω άλλα 36 και στην συμπληρωματική έξοδο nQ του ασύγχρονου D Flip Flop.

Τέλος, προσθέτοντας στο config file τα υπόλοιπα πεδία και κάνοντας τους κατάλληλους ελέγχους, θα τυπωθούν τα αρχικά πεδία του DFFRS όπως φαίνεται στο παρακάτω στιγμιότυπο

```

729 cell("DFFRS_X1") {
730
731     ff ("IQ","nIQ") {
732         next_state: "D"
733         clocked_on: "Clk"
734         preset: "Pre"
735         clear: "Clr"
736     }
737     pin("D") {
738
739         direction: input;
740         capacitance: 15.4;
741     }
742     pin("Clk") {
743
744         direction: input;
745         capacitance: 15.4;
746     }
747     pin("Pre") {
748
749         direction: input;
750         capacitance: 15.4;
751     }
752     pin("Clr") {
753
754         direction: input;
755         capacitance: 15.4;
756     }
757     pin("Q") {
758
759         direction: output;
760         function: IQ;
761     }
762     pin("nQ") {
763
764         direction: output;

```

ΜΕΛΛΟΝΤΙΚΕΣ ΠΡΟΣΘΗΚΕΣ

Προφανώς, το πρώτο πράγμα που πρέπει να γίνει είναι να ολοκληρωθεί είναι ο χαρακτηρισμός των timing και constraint arcs του DFFRS

Από κει και πέρα, το εργαλείο θα μπορούσε, προσθέτοντας και άλλα μοντέλα, να δημιουργήσει και άλλα standard cells, κάνοντας τις κατάλληλες διορθώσεις στην rython, εάν χρειαστεί. Τα δεδομένα των νέων πυλών θα εισάγονται στο config file ακολουθώντας το ίδιο μοτίβο με τα ήδη δημιουργημένα standard cells, έτσι ώστε να διαβαστούν σωστά από το εργαλείο και να τυπωθεί σωστά η βιβλιοθήκη

Τέλος, ίσως να ήταν δυνατό να μειωθεί και η πολυπλοκότητα του εργαλείου, καθώς χρησιμοποιούνται αρκετά nested loops με την πολυπλοκότητα του προγράμματος να είναι $O(n^5)$.